

ECN 753: Industrial Organization

Homework 2

Replicating Rust (1987)

Andres A. Saenz Guzman^{*†}

November 6, 2024

1 Rust (1987)

1.1 Nested Fixed-Point Method (NFXP)

After multiple attempts in building a quasi-functional code, I managed to produce a code that ran smoothly but produced the wrong results ($\theta_{11} = 3.5$ and $RC = 7.4$), hence the quasi. After studying Erhan's code, hence the mention at the beginning, I tried to see the differences between both codes and make the necessary adjustments to mine. This resulted in many hours of unfruitful work as my code kept yielding the same output, although the logic in both seemed to be the same. As the deadline approached I got desperate enough to ask GPT about the difference in both codes. Its response was not expected: the codes were similar in logic with some minor differences that, according to the A.I. should not have a mayor impact. Being doubtful of this response, I asked it to adjust my original code accordingly, which produced even worse, even nonsensical results.

Anyways, sorry for the long story. Here are the results from the code using different optimizers in the minimization function.

Table 1: Estimation Results

Optimizer	RC	θ_{11}
L-BFGS-B	10.0749747	2.2930975
Nelder-Mead	10.0750200	2.2930749
Powell	10.6267141	2.5053880
TNC	10.3304416	2.3928359
COBYLA	10.0721553	2.2918425
SLSQP	10.0748748	2.2929661

^{*}Arizona State University, Department of Economics.

[†]Many thanks to Ömer Erhan Erbis for his help with coding hints.

1.2 Conditional Choice Probability (CCP) Method

The CCP method has the benefit of simplicity, making it less computationally expensive than the NFXP method because it avoids iterating over the Bellman equation. However, this approach is limited to the quality and quantity of data, since it assumes that available data is large enough to accurately represent the true probabilities. In the rest of this document are the three steps to use the CCP method with an example of a non-operational code:

1. Estimate CCP from data

```
1 import numpy as np
2 import pandas as pd
3 from scipy.optimize import minimize
4 from sklearn.linear_model import LogisticRegression
5
6 data = pd.read_csv('Data/group_4.csv')
7
8 # Logistic regression model for replacement probability as a function of mileage
9 logit_model = LogisticRegression()
10 X = data[['mileage']].values # Mileage is the explanatory variable
11 y = data['decision'].values # Replacement decision (1 if replaced, 0 otherwise)
12 logit_model.fit(X, y)
13 data['ccp_replace'] = logit_model.predict_proba(X)[:, 1] # Predicted CCP for
    replacement
```

2. Use a simpler choice model and compute expected utilities for each choice

```

1 # Model parameters to be estimated
2 initial_params = [2, 2] # Initial guesses for parameters [theta, c_replace]
3
4 beta = 0.9999 # Discount factor
5
6 # Define the log-likelihood function using the CCP estimates
7 def log_likelihood_ccp(params, data):
8     theta, c_replace = params
9     log_likelihood_value = 0
10
11     # Iterate over each bus
12     for _, row in data.iterrows():
13         mileage = row['mileage']
14         replacement_decision = row['decision']
15         ccp_replace = row['ccp_replace']
16
17         # Calculate costs and values based on mileage
18         maint_cost = 0.001 * theta * mileage
19         replace_cost = c_replace
20
21         # Approximate value function using CCP
22         prob_replace = ccp_replace
23         prob_continue = 1 - prob_replace
24
25         # Add small constant to avoid log(0)
26         prob_replace = max(prob_replace, 1e-10)
27         prob_continue = max(prob_continue, 1e-10)
28
29         # Calculate log-likelihood contribution based on the observed decision
30         if replacement_decision == 1:
31             log_likelihood_value += np.log(prob_replace)
32         else:
33             log_likelihood_value += np.log(prob_continue)
34
35         # Debugging output
36         print(f"Params: theta={theta}, c_replace={c_replace}, Log-likelihood={-log_likelihood_value}")
37
38     return -log_likelihood_value # Negative log-likelihood for minimization
39
40

```

3. Optimize parameters by maximizing the likelihood of observed choices given the computation above

```

1 # Estimate the parameters using maximum likelihood with CCPs
2 result = minimize(log_likelihood_ccp, initial_params, args=(data,), method='
    Powell', options={'disp': True})
3 estimated_params = result.x
4 print("Estimated parameters using CCP method:", estimated_params)

```